📝 Week 2 Recap: Control Flow & Vectorized Layouts

Course Block: CP352301 Script Programming (Semester 1/2026)

Core Framework Reference: *Week 2: Control Flow: Conditionals & p2. List Comprehention*
AI Governance Context: Evaluated and structured with AI ethics, explainable AI, and responsible parameters in mind.

---

1. Conceptual Deep-Dive: Decision Trees & Logic Inversions

Programs achieve dynamic flow state routing by checking relational conditions against runtime variables, resulting in primitive `True` or `False` Boolean states.

- **Relational Scoping:** Standard equality (`==`, `!=`) and directional thresholds (`<`, `>`, `<=`, `>=`) serve as structural benchmarks.
- **Logical Operations:** Short-circuiting evaluation paths (`and`, `or`, `not`) allow cascading logic gates.
- Using `and` forces both expressions to settle as `True`.
- Using `or` satisfies the condition if at least one side is verified.
- Using `not` performs an inversion check on any boolean evaluation.
- **Indentation Syntax Rule:** Unlike bracket-enclosed languages, Python enforces structural indentation (blocks of whitespace metrics) to group scope boundaries. Failing to maintain uniform tabs or 4-space alignments results in an immediate `IndentationError`.

---

2. Mutually Exclusive Chains vs. Nested Conditionals

- **Sequential Mutually Exclusive Routing (`if-elif-else`):** The engine checks conditions from top to bottom. Once a condition resolves as `True`, Python executes its nested indentation code block and **immediately short-circuits out** of the entire chain.

```python
# Source Document Verification: Lecture Grading Scale
score = 84.5
if score >= 90:
```

```
    grade = "A"
elif score >= 80:  # Evaluates True. The engine assigns variable and exits.
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"
```

- **Multi-Gate Constraints (Nested Lines):** Evaluates separate verification axes by nesting conditional gates inside one another. Internal pathways will execute *only* if the parent gate resolves to `True`.

```python
# Source Document Verification: Membership Discount Gates
is_member = "yes"
purchase_amount = 125.0

if is_member == "yes":             # Parent Gate (Resolved)
    if purchase_amount >= 100:     # Child Gate (Resolved)
        discount = purchase_amount * 0.15
    else:
        discount = purchase_amount * 0.05
else:
    discount = 0.0
```

---

3. Structural Optimization: Inline Comprehensions

List comprehensions provide optimized, single-line syntactical constructs to instantiate lists out of existing iterables, evaluating elements immediately inside an internal allocation stream.

$$\text{Syntax Structure: } \mathbf{[};\text{expression};\mathbf{for};\text{element};\mathbf{in};\text{iterable};\mathbf{if};\text{condition};\mathbf{]}$$

- **Mathematical Scalar Transformation:** Instead of initializing an empty list and running a multi-line `for` loop to manually `.append()` elements, list comprehensions execute inline at the C-level speed.

```python
# Slide 6 Snippet: Square integers up to 9 immediately
squares = [x**2 for x in range(10)]
# Output Matrix: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

- **Combined Relational Transformations:** Filtering datasets inside comprehensions using advanced logical operators helps optimize parsing functions:

```python
# Slide 6 Snippet: Multiples of 3 but NOT multiples of 6 under 50
filtered_multiples = [x for x in range(50) if x % 3 == 0 and x % 6 != 0]
# Output Vector: [3, 9, 15, 21, 27, 33, 39, 45]
```

🌱 Week 2 Seed: The Continuity Hook for Week 3

While **Week 2** focused on immediate, memory-heavy list transformations and single decision paths, real-world scripting requires managing open-ended data streams, handling volatile connections, and unpacking complex, multi-dimensional structures.

The Limits of Immediate Evaluation

List comprehensions consume computing memory proportional to their output length because they generate the entire list structure in memory at once. If you try to parse a massive stream of millions of automated network server logs, using a list comprehension could spike memory usage and crash the system runtime.

🔗 The Continuity Hook to Week 3

To prepare for **Week 3: Loops, Iteration Control, and Dynamic Collections**, consider these three upcoming architectural concepts:

1. **Indeterminate Automation (`while` streams):** How can a script safely check for live system log drops indefinitely without causing an infinite execution loop or crashing memory stacks?
2. **Lazy Evaluation vs. Materialized Memory:** How can we use parenthetical generator expressions `(x2 for x in data)` to process data elements one at a time, keeping memory footprint constant at $O(1)$ instead of memory scaling at $O(N)$?
3. **Collection Unpacking Loops:** How can a script traverse structured tuple coordinate indices natively inside loop declarations to unpack composite records on the fly?

Python

```python
# --- SNEAK PEEK CRITERIA FOR WEEK 3 ---
# Unpacking composite network coordinate tuples inside iteration streams
active_nodes = [("Node_A", "10.0.0.1"), ("Node_B", "192.168.1.1")]

for label, ip_address in active_nodes:
    print(f"Targeting System Destination: {label} via Endpoint Address: {ip_address}")
```

July 3, 2026 Submission Target Reminder: Ensure your local workspace contains both `labs/integrated_sandbox.py` and your structural learning notebook `learning_log.ipynb` fully synchronized before the class repository evaluation deadline.